

COMPUTER PROGRAMMING FOR PETROLEUM ENGINEERS

PROJECT 2 - Algorithm Design & Comparison

INDEX NUMBER :4072924

Part A — Two Search Algorithms (4 marks)

(a) Pseudocode

Linear Search

INPUT array, n

SET maxTemp TO array[0]

SET maxIndex TO 0

SET i TO 1

SET comparisons TO 0

WHILE $i < n$

 SET comparisons TO comparisons + 1

 IF array[i] > maxTemp THEN

 SET maxTemp TO array[i]

 SET maxIndex TO i

 END IF

 SET i TO i + 1

END WHILE

OUTPUT maxTemp

OUTPUT maxIndex

OUTPUT comparisons

Sort-then-Select

INPUT array, n

SET comparisons TO 0

SET i TO 0

WHILE $i < n - 1$

```

SET j TO 0
WHILE j < n - i - 1
    SET comparisons TO comparisons + 1
    IF array[j] > array[j + 1] THEN
        SET temp TO array[j]
        SET array[j] TO array[j + 1]
        SET array[j + 1] TO temp
    END IF
    SET j TO j + 1
END WHILE
SET i TO i + 1
END WHILE
SET maxTemp TO array[n - 1]
OUTPUT maxTemp
OUTPUT comparisons

```

(b) Trace — Linear Search on [87, 143, 52, 201, 98, 76]

i	array[i]	Comparison	maxTemp	maxIndex
—	—	—	87	0
1	143	143 > 87 → Yes	143	1
2	52	52 > 143 → No	143	1
3	201	201 > 143 → Yes	201	3
4	98	98 > 201 → No	201	3
5	76	76 > 201 → No	201	3

Total
comparisons (6

elements): 5

Trace — Sort-then-Select on [87, 143, 52, 201, 98, 76]

Pass	Array after pass	Comparisons	Running total
1	87, 52, 143, 98, 76, 201	5	5
2	52, 87, 98, 76, 143, 201	4	9
3	52, 87, 76, 98, 143, 201	3	12
4	52, 76, 87, 98, 143, 201	2	14
5	52, 76, 87, 98, 143, 201	1	15

Total comparisons (6 elements): 15 → final max = 201 (last element after sort)

(c) Comparisons on the full 20-element list

Algorithm	Comparisons
Linear Search	19
Sort-then-Select	190

Part B — Alarm Detection Algorithm

INPUT array, n, threshold

SET count TO 0

SET sumTemp TO 0

SET i TO 0

WHILE i < n

 IF array[i] > threshold THEN

 SET indices[count] TO i

 SET sumTemp TO sumTemp + array[i]

 SET count TO count + 1

 END IF

 SET i TO i + 1

END WHILE

SET averageTemp TO sumTemp / count

OUTPUT count

OUTPUT indices

OUTPUT averageTemp

(b) Complexity Classification

Time complexity: $O(n)$, linear time. One pass through the array, once for each element (20 comparisons for this array) regardless of the number of sensors that set off the alarm.

(c) Trace on the full 20-element list (threshold = 180°C)

i	array[i]	array[i] > 180?	count	sumTemp
0	87	No	0	0
1	143	No	0	0
2	52	No	0	0
3	201	Yes $\rightarrow$ indices[0] = 3	1	201
4	98	No	1	201
5	76	No	1	201
6	165	No	1	201
7	44	No	1	201
8	119	No	1	201
9	88	No	1	201
10	203	Yes $\rightarrow$ indices[1] = 10	2	404
11	61	No	2	404
12	177	No	2	404
13	93	No	2	404
14	55	No	2	404
15	142	No	2	404

16	188	Yes → indices[2] = 16	3	592
17	72	No	3	592
18	110	No	3	592
19	135	No	3	592

Final results: count = 3, indices = [3, 10, 16], average temperature = $592 / 3 = 197.33^{\circ}\text{C}$

Part C — Efficiency Analysis

Algorithm	Time Complexity	Operations (n=20)	Operations (n=200)	Operations (n=20,000)	Best Suited Practical Scenario
Linear Search	O(n)	19	199	19,999	One-off lookup of a single maximum reading; no need for the data to be ordered
Sort-then-Select	O(n²)	190	19,900	199,990,000	When the sorted data itself is needed beyond the max — ranked reports, median, top-k sensor readings

Alarm Detection	O(n)	20	200	20,000	Real-time threshold/safe ty monitoring across large, continuously updated sensor networks
----------------------------	-------------	-----------	------------	---------------	--

Summary note:

At 20,000 readings, Sort-then-Select makes about 200 million comparisons, while the two $O(n)$ algorithms each make 20,000 comparisons — a difference of $\sim 10,000\times$ gap that shows the reason sorting a whole dataset just so to get one maximum is inefficient a